

Events & Messenger

L'asynchronicité dans Symfony

Table des matières

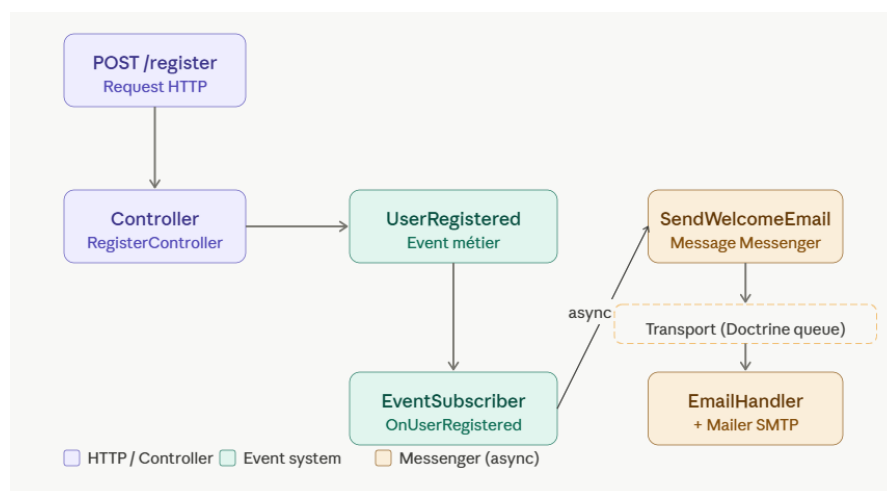
1. Introduction.....	2
Créer un User.....	3

1. Introduction

Symfony possède son propre système d'écoute d'évènement, semblables aux EventListener de Javascript. Problème : comment faire en sorte qu'un évènement soit « écouté » puisque le code côté serveur n'est pas interprété par le navigateur et donc qu'on n'a pas de détection du comportement ?

Ce n'est pas exactement comme Javascript que ça fonctionne effectivement, mais on va quand même pouvoir répliquer son comportement **asynchrone**

Nous allons mettre en place un nouveau projet pour illustrer cet aspect de Symfony, nous l'appellerons NotifApp et voici son architecture :



- Créez votre projet : `symfony new notifapp --version="7.4.*" --webapp`
- Copiez ces deux fichiers dans respectivement **compose.yaml** et **.env.local**, et videz le contenu de **compose.override.yaml**.

```
services:
  database:
    image: mariadb:11
    environment:
      MARIADB_ROOT_PASSWORD: root
      MARIADB_DATABASE: notifapp
      MARIADB_USER: notifapp_user
      MARIADB_PASSWORD: notifapp_pwd
    volumes:
      - notifapp_db_data:/var/lib/mysql
    ports:
      - "3306:3306"
  mailpit:
    image: axllent/mailpit
    ports:
      - "1025:1025" # SMTP
      - "8025:8025" # Interface web
volumes:
  notifapp_db_data:
```

```
DATABASE_URL="mysql://notifapp_user:notifapp_pwd@127.0.0.1:3306/notifapp?serverVersion=mariadb-11.0.0"
```

Enfin, dans le fichier .env, on va aller remplacer la valeur de la clé MAILER_DSN et MESSENGER_TRANSPORT_DSN :

```
MESSENGER_TRANSPORT_DSN=doctrine://default?auto_setup=true
```

```
###> symfony/mailer ###  
MAILER_DSN=smtp://localhost:1025  
###< symfony/mailer ###
```

Créer un User

php bin/console make:user puis choix par défaut à toutes les questions. On va ensuite rajouter une propriété **username** au User en faisant php bin/console make:entity User : **string, 100, not null**

Exécutez votre migration après

Enfin, on va utiliser le makerBundle pour créer RegisterController.php :

```
php bin/console make:controller Register
```

On est prêts

Maintenant on va pouvoir aborder LE BUT DU JEU : Déclencher un envoi d'e-mail lorsqu'un utilisateur s'inscrit

Nous allons devoir créer des fichiers à la main ici. On va créer le dossier Event dans le dossier src et y créer le fichier **UserRegisteredEvent.php**

C'est un fichier dans lequel on va définir quelles données doivent parvenir à l'évènement lorsqu'il est détecté, un peu comme un DTO. Voici le code à coller dans le fichier :

```
<?php  
// src/Event/UserRegisteredEvent.php  
namespace App\Event;  
  
class UserRegisteredEvent  
{  
    public function __construct(  
        private readonly string $email,  
        private readonly string $username,  
    ) {}  
  
    public function getEmail(): string { return $this->email; }  
    public function getUsername(): string { return $this->username; }  
}
```

Nous allons créer maintenant un fichier **SendWelcomeEmailMessage.php** dans un nouveau dossier **Message** :

```
<?php
// src/Message/SendWelcomeEmailMessage.php
namespace App\Message;

final class SendWelcomeEmailMessage
{
    public function __construct(
        public readonly string $email,
        public readonly string $username,
    ) {}
}
```

Nous allons créer maintenant un fichier **SendWelcomeEmailHandler.php** dans un nouveau dossier **MessageHandler** :

```
<?php
// src/MessageHandler/SendWelcomeEmailHandler.php
namespace App\MessageHandler;

use App\Message\SendWelcomeEmailMessage;
use Symfony\Bridge\Twig\Mime\TemplatedEmail;
use Symfony\Component\Mailer\MailerInterface;
use Symfony\Component\Messenger\Attribute\AsMessageHandler;

#[AsMessageHandler]
class SendWelcomeEmailHandler
{
    public function __construct(private readonly MailerInterface $mailer) {}

    public function __invoke(SendWelcomeEmailMessage $message): void
    {
        $email = (new TemplatedEmail())
            ->from('noreply@notifapp.com')
            ->to($message->email)
            ->subject('Bienvenue sur NotifApp !')
            ->htmlTemplate('emails/welcome.html.twig')
            ->context(['username' => $message->username]);

        $this->mailer->send($email);
    }
}
```

Nous allons ensuite créer **UserRegisteredSubscriber.php** dans un nouveau dossier **EventSubscriber** :

```
<?php
// src/EventSubscriber/UserRegisteredSubscriber.php
namespace App\EventSubscriber;

use App\Event\UserRegisteredEvent;
use App\Message\SendWelcomeEmailMessage;
use Symfony\Component\EventDispatcher\EventSubscriberInterface;
use Symfony\Component\Messenger\MessageBusInterface;

class UserRegisteredSubscriber implements EventSubscriberInterface
{
    public function __construct(private readonly MessageBusInterface $bus) {}

    public static function getSubscribedEvents(): array
    {
        return [UserRegisteredEvent::class => 'onUserRegistered'];
    }

    public function onUserRegistered(UserRegisteredEvent $event): void
    {
        $this->bus->dispatch(new SendWelcomeEmailMessage(
            email: $event->getEmail(),
            username: $event->getUsername(),
        ));
    }
}
```

On se prépare un template :

```
{# templates/emails/welcome.html.twig #}
<!DOCTYPE html>
<html>
<body>
    <h1>Bonjour {{ username }} !</h1>
    <p>Votre compte a bien été créé sur NotifApp.</p>
    <p>Merci de nous rejoindre.</p>
</body>
</html>
```

Et on lance le worker pour pouvoir voir ce qu'il se passe en temps réel :

```
php bin/console messenger:consume async --vv
```

Ce worker est relié à la table `messenger_messages` que Symfony crée automatiquement. IL va consulter ces messages et les afficher en temps réel dans le terminal. Il ne nous reste plus qu'à tester si tout fonctionne en exécutant cette commande dans un autre terminal :

```
curl -X POST https://localhost:8000/register -H "Content-Type: application/json" -d '{"email":"alice32@test.com","username":"alice32","password":"motdepasse"}'
```

WHAT THE FUCK IS THAT



Oui, on va avoir besoin d'un peu d'explications.

Les quatre fichiers

UserRegisteredEvent.php — "quelque chose s'est passé"

C'est une simple annonce, un fait accompli. Il dit au reste de l'application : *"un utilisateur vient de s'inscrire, voilà ses données si quelqu'un en a besoin."*

Il ne fait rien par lui-même. C'est un objet passif qui transporte de l'information. Symfony le dispatche dans tout le système, et tous ceux qui veulent réagir peuvent l'écouter.

La distinction importante : un événement parle du **passé** (`UserRegistered`, pas `SendEmail`). Il décrit ce qui s'est passé, pas ce qu'il faut faire ensuite. C'est intentionnel — le contrôleur ne sait pas et ne doit pas savoir ce que le reste de l'application va faire de cette information.

UserRegisteredSubscriber.php — "je réagis à la chose qui s'est passé"

C'est l'écouteur. Il dit à Symfony : *"quand tu vois passer un UserRegisteredEvent, appelle-moi."*

Pensez à ce que fait la méthode **subscribe** chez Angular HttpClient, c'est la même chose 😊

Son rôle est de faire le lien entre l'événement (ce qui s'est passé) et l'action à déclencher. Mais — et c'est le point clé — il ne fait pas l'action lui-même. Il se contente de **déposer un message dans la queue Messenger** et de rendre la main immédiatement grâce à la méthode **dispatch**.

Pourquoi ne pas envoyer l'email directement ici ? Parce que ça bloquerait la réponse HTTP le temps que le serveur mail réponde. En déposant un message dans la queue, le subscriber termine son travail en quelques microsecondes.

SendWelcomeEmailMessage.php — "la tâche à exécuter plus tard"

C'est le message glissé dans la file d'attente. Il contient uniquement les données nécessaires pour accomplir la tâche plus tard : l'email et le nom d'utilisateur.

Il ressemble beaucoup à l'événement, et c'est là que la confusion est fréquente. La différence conceptuelle : l'événement parle du **passé** (UserRegistered), le message parle du **futur** (SendWelcomeEmail). **L'événement est une observation, le message est une action différée.**

Concrètement, ce fichier est aussi ce qui est **sérialisé et stocké en base de données** dans la table messenger_messages. Il doit donc rester un objet simple, sans dépendances vers des services Symfony.

SendWelcomeEmailHandler.php — "j'exécute la tâche"

C'est le seul fichier qui fait quelque chose de concret. Il est exécuté par le worker dans le second terminal, de manière totalement indépendante de la requête HTTP qui a tout déclenché.

Il reçoit le message, accède au Mailer, construit l'email et l'envoie. C'est ici que vivent les dépendances lourdes (le MailerInterface), justement parce que ce code ne s'exécute pas pendant la requête web.

Le tableau récapitulatif

Fichier	Rôle	Moment d'exécution	Fait quelque chose ?
UserRegisteredEvent	Annonce un fait	Pendant la requête HTTP	Non, passif
UserRegisteredSubscriber	Réagit à l'annonce	Pendant la requête HTTP	Juste dépose un message
SendWelcomeEmailMessage	Tâche mise en attente	Stocké en BDD	Non, passif
SendWelcomeEmailHandler	Exécute la tâche	Dans le worker (async)	Oui, envoie l'email

La ligne de séparation importante est entre les deux premiers (qui s'exécutent **pendant** la requête) et les deux derniers (qui s'exécutent **après**, dans le worker). C'est cette séparation qui rend l'envoi d'email asynchrone.

Et donc quel intérêt ?

En dissociant certaines actions de la requête HTTP — comme l'envoi d'email, la génération de PDF ou la synchronisation avec un service externe — on permet à l'utilisateur d'obtenir une réponse immédiate sans attendre que ces tâches soient terminées. Le statut de la réponse reflète uniquement le succès de l'action principale (ici, la création du compte), pas celui des effets de bord déclenchés dans la foulée. C'est le principe fondamental de l'architecture asynchrone : **dissocier ce qui doit être fait maintenant de ce qui peut être fait plus tard.**